

1. What is Logging in ASP.NET Core?

Logging is the process of recording **application events** (information, warnings, errors, and critical failures) so that developers and operators can understand **what happened and why**.

Typical Questions Logging Answers

- Did an error occur?
- What exception was thrown?
- What input caused the failure?
- Did a database call fail?

Log Levels (Built-in)

Level	Purpose
Trace	Very detailed, diagnostic info
Debug	Development diagnostics
Information	Normal application flow
Warning	Unexpected but non-fatal issues
Error	Failures that need attention
Critical	Application-stopping failures

2. Logging Example (ASP.NET Core Web API)

Step 1: Built-in Logging (No Extra Packages)

ASP.NET Core provides `ILogger<T>` out of the box.

Controller Example

```
[ApiController]
```

```
[Route("api/products")]
```

```
public class ProductsController : ControllerBase
```

```
{  
    private readonly ILogger<ProductsController> _logger;  
  
    public ProductsController(ILogger<ProductsController> logger)  
    {  
        _logger = logger;  
    }  
  
    [HttpGet("{id}")]  
    public IActionResult Get(int id)  
    {  
        _logger.LogInformation("Fetching product with Id {ProductId}", id);  
  
        if (id <= 0)  
        {  
            _logger.LogWarning("Invalid product id received: {ProductId}", id);  
            return BadRequest();  
        }  
  
        try  
        {  
            // Simulate exception  
            throw new Exception("Database connection failed");  
        }  
        catch (Exception ex)  
        {  

```

```
        _logger.LogError(ex, "Error occurred while fetching product {ProductId}", id);
        return StatusCode(500, "Internal Server Error");
    }
}
}
```

What This Produces

- **Information** log for request flow
 - **Warning** for invalid input
 - **Error** log with exception stack trace
-

3. What is Tracing?

Tracing tracks a **single request across multiple components** and services using a **TraceId / CorrelationId**.

Typical Questions Tracing Answers

- How long did this request take?
 - Which service caused the delay?
 - Where did the request fail?
 - How did the request flow across microservices?
-

4. Tracing vs Logging (Key Difference)

Aspect	Logging	Tracing
Focus	Events	Request lifecycle
Scope	Single app	Multiple services
Identifiers	Message-based	TraceId / SpanId

Aspect Logging Tracing

Example “DB connection failed” “Request took 2.3s across 3 services”

5. Tracing Example (ASP.NET Core – Simple)

Step 1: Enable Activity (Built-in)

ASP.NET Core already supports System.Diagnostics.Activity.

Middleware for Correlation ID

```
public class CorrelationIdMiddleware
{
    private readonly RequestDelegate _next;
    private const string HeaderName = "X-Correlation-Id";

    public CorrelationIdMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        var correlationId = context.Request.Headers.ContainsKey(HeaderName)
            ? context.Request.Headers[HeaderName].ToString()
            : Guid.NewGuid().ToString();

        context.Response.Headers[HeaderName] = correlationId;
    }
}
```

```
using (Activity activity = new Activity("HTTP Request"))
{
    activity.SetIdFormat(ActivityIdFormat.W3C);
    activity.Start();
    activity.AddTag("correlation.id", correlationId);

    await _next(context);

    activity.Stop();
}
}
```

Step 2: Register Middleware

```
app.UseMiddleware<CorrelationIdMiddleware>();
```

6. Logging + Tracing Together (Best Practice)

```
_logger.LogInformation(
    "Processing order {OrderId} with TraceId {TraceId}",
    orderId,
    Activity.Current?.TraceId.ToString());
```